챕터 9. 컨베이어 벨트 위에 올려 둔다. RabbitMQ + Spring AMQP

이번 챕터가 끝나면

- 저장 API 한 번에 여러 저장소(RDB·ES·파일·외부 시스템)를 직렬로 묶을 때 생기는 **이중 저장 불일치**와 **응답 지연**을, 컨베이어 벨트 비유로 한 장면에서 잡습니다
- Docker Compose 로 `rabbitmq:3-management` 를 띄우고, 관리 UI(`15672`) 에서 Exchange · Queue · Binding 을 눈으로 확인합니다
- Spring Boot 에 `spring-boot-starter-amqp` 를 얹어, Producer 는 `RabbitTemplate.convertAndSend()` 로 메시지를 발행하고 Consumer 는 `@RabbitListener` 로 큐를 구독합니다
- GitHub README.md 의 변경을 `@Scheduled` 폴링으로 감지하고, Direct Exchange(`github.events`) + Routing Key(`readme.changed`) 로 Queue(`repo-updates`) 에 메시지를 싣는 전체 파이프라인을 손끝으로 조립합니다
- 통합 테스트에서 GitHub 에 커밋을 올리면, Producer 로그에 "변경 감지", Consumer 로그에 "메시지 수신", 그리고 로컬 `README.md` 와 `backups/` 폴더까지 자동으로 따라오는 것을 직접 확인합니다
- 마지막 절에서 Polling → Webhook 으로 바꿀 때 구조가 어떻게 바뀌는지 짚고, 스프링 레퍼런스 아홉 기둥을 한 번에 되돌아보겠습니다

이번 챕터는 "공장에서 생산자가 완성품을 컨베이어 벨트에 올려 두면, 소비자가 원할 때 집어가는 이야기"입니다

챕터 8 끝에서 오픈이는 수첩에 세 방향을 적어 뒀습니다. 동기 보상·배치 재색인·메시지 큐. 이 챕터에서는 세 번째 방향, 메시지 큐를 직접 얹습니다. 다만 ES 동기화를 바로 큐에 태우지 않고, 같은 패턴이 들어간 더 단순한 문제를 먼저 연습합니다. 팀장이 던진 주제는 "GitHub 의 README 가 바뀌면 우리 서비스의 로컬 파일도 따라 바뀌게 해 봐요" 입니다. 관찰 서버(Producer) 하나, 반영 서버(Consumer) 하나, 그 사이를 잇는 RabbitMQ 하나. 세 프로세스를 띄워 놓고, GitHub 에 커밋 한 번을 누르는 것만으로 로컬 파일이 따라 변하는 그림을 끝까지 따라갑니다. 마지막에는 이 9장 동안 오픈이가 지나온 **인증·세션·파일·실시간·스트리밍·검색·이벤트** 여섯 기둥을 한 번에 되짚겠습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

챕터 9 는 Spring Boot 앱 **두 개**와 별도의 RabbitMQ 컨테이너 하나, 그리고 GitHub 에 따로 만드는 **관찰 대상 레포** 하나를 합쳐 구성합니다. Producer 와 Consumer 가 분리된 프로세스이기 때문에 레포도 두 개입니다.

레포	용도	주소
rabbitmq-docker	RabbitMQ 브로커(Docker Compose, 관리 UI 포함)	`github.com/metacoding-11-spring-reference/rabbitmq-docker`
rabbitmq-producer	관찰 서버. GitHub 폴링 + 메시지 발행	`github.com/metacoding-11-spring-reference/rabbitmq-producer`
rabbitmq-consumer	수신 서버. 큐 구독 + 로컬 `README.md` 반영	`github.com/metacoding-11-spring-reference/rabbitmq-consumer`

터미널에서 세 레포를 각각 클론합니다.

터미널 실습 레포 클론

```
git clone https://github.com/metacoding-11-spring-reference/rabbitmq-docker.git
git clone https://github.com/metacoding-11-spring-reference/rabbitmq-producer.git
git clone https://github.com/metacoding-11-spring-reference/rabbitmq-consumer.git
```

파일 구조는 이렇습니다. 세 레포를 나란히 배치한 형태입니다.

rabbitmq-* 디렉토리

```
rabbitmq-docker/
└── docker-compose.yml                                  # [실습] RabbitMQ + 관리 UI

rabbitmq-producer/
├── build.gradle                                        # [실습] spring-boot-starter-amqp 의존성
└── src/main/
    ├── java/com/rabbitmq/producer/
    │   ├── github/
    │   │   └── GitHubClient.java                       # [실습] GitHub REST 호출
    │   ├── scheduler/
    │   │   └── PollingScheduler.java                   # [실습] @Scheduled 변경 감지
    │   ├── rabbit/
    │   │   └── RabbitProducer.java                     # [실습] RabbitTemplate.convertAndSend
    │   └── dto/
    │       └── RabbitDTO.java                          # [실습] 메시지 계약
    └── resources/
        └── application.properties                      # [실습] GitHub · RabbitMQ 설정

rabbitmq-consumer/
├── build.gradle                                        # [실습] spring-boot-starter-amqp 의존성
```

```
└── src/main/
    ├── java/com/rabbitmq/consumer/
    │   ├── config/
    │   │   └── RabbitConfig.java                          # [실습] Queu
e 빈 · JSON 컨버터
    │   ├── listener/
    │   │   └── RabbitConsumer.java                        # [실습] @Rabbi
tListener + 파일 반영
    │   └── dto/
    │       └── RabbitDTO.java                             # [실습] Produc
er 와 동일한 필드
    └── resources/
        └── application.properties                         # [실습] Rabbit
MQ 설정
└── README.md                                              # [참고] 수신 결
과가 덮어쓰이는 파일
```

> **RabbitMQ 는 Spring 프로세스 밖에 있는 별도 서버입니다.** Redis · Elasticsearch 와 마찬가지로, 앱을 띄우기 전에 RabbitMQ 프로세스가 먼저 떠 있어야 합니다. 이 실습은 Docker Compose 로 RabbitMQ 한 컨테이너만 올리고, Producer 와 Consumer 는 각자의 로컬 Gradle 앱으로 띄웁니다.

2. 실습 환경 구축

터미널 실습 환경 확인

```
docker --version
docker compose version
java --version
```

Docker Desktop 이 떠 있어야 하고, JDK 17 이상이 설치돼 있어야 합니다. RabbitMQ 이미지는 `rabbitmq:3-management` 를 씁니다. AMQP(`5672`)·관리 UI(`15672`) 두 포트가 비어 있어야 합니다.

3. GitHub 관찰 대상 레포 준비

Producer 가 볼 대상 레포를 따로 하나 만들어야 합니다. 이 실습용 레포가 바로 "변경을 일으키는 곳" 입니다.

1. 본인 GitHub 계정에서 새 레포지토리 생성
2. 이름: `config-readme`
3. 로컬에서 `README.md` 파일 생성 후 커밋·푸시

이 레포의 `README.md` 가 바뀔 때마다 Producer 가 감지합니다.

4. 사용할 구성 요소

이번 챕터에 새로 얹히는 재료는 여덟 가지입니다.

재료	역할
`rabbitmq:3-management`	RabbitMQ 브로커 본체 + 관리 UI. AMQP `5672` · UI `15672`
`spring-boot-starter-amqp`	Spring AMQP. `RabbitTemplate` · `@RabbitListener` · `ConnectionFactory` 기본 제공
`RabbitTemplate.convertAndSend`	Producer 에서 Exchange + Routing Key + payload 로 메시지 발행
`@RabbitListener(queues = "...")`	Consumer 에서 Queue 를 구독해 메시지 수신
`Jackson2JsonMessageConverter`	payload JSON ↔ DTO 변환. 등록 안 하면 역직렬화 실패
`@Scheduled(fixedRateString = ...)`	Producer 폴링 주기. `application.properties` 에서 주입
`RestTemplate`	GitHub REST API 호출(커밋 목록·raw 파일)
Direct Exchange	Routing Key 가 **정확히 일치** 할 때만 Queue 로 전달. 오타를 바로 발견

5. 실습 순서

이번 챕터는 이 순서로 흘러갑니다.

1. `9.1` . 챕터 8 의 수첩을 다시 펼친다
2. `9.2` . 동기 호출 한 줄로 엮었을 때의 실패 (Try/Fail)
3. `9.3` . 공장 컨베이어 벨트와 우체국 접수대 (큐의 비유)
4. `9.4` . Docker 로 RabbitMQ 띄우기
5. `9.5` . Producer · Consumer 공통 토폴로지 고정 (Exchange · Queue · Routing Key)
6. `9.6` . GitHub 파일 폴링 설계 (sha 비교 + 중복 발행 방지)

7. `9.7`. 관찰 서버 구현 (Polling → Publish)
8. `9.8`. 수신 서버 구현 (Listener → 파일 반영)
9. `9.9`. 통합 테스트. GitHub 커밋 한 번으로 세 프로세스가 한 번에 움직인다
10. `9.10`. Polling 을 Webhook 으로 바꿔 본다 (실무 확장)
11. `9.11`. 아홉 기둥을 돌아본다

1~2 에서 챕터 8 이 남긴 문제와 동기 호출의 한계를 짚고, 3 에서 메시지 큐의 비유를 고정합니다. 4~6 에서 RabbitMQ 와 양쪽 앱의 토폴로지를 맞추고, 7~8 에서 Producer · Consumer 를 한 조각씩 조립합니다. 9 에서 GitHub 커밋 하나로 세 프로세스가 같이 움직이는 것을 눈으로 확인하고, 10 에서 Polling 을 Webhook 으로 바꿀 때의 구조 변화를 살펴본 뒤, 11 에서 이 책 전체를 닫습니다.

9.1 "그래서, 그 큐는 뭐부터 해요?"

챕터 8 을 닫은 다음 날 오전이었습니다. 오픈이는 커피를 들고 자리에 앉았는데, 팀장이 모니터 옆으로 의자를 끌고 왔습니다.

팀장 : "어제 수첩에 세 방향 적어 뒀잖아요. 그중에 세 번째."

오픈이 : "메시지 큐요. 저장은 RDB 만 하고, ES 는 구독자가 따로 맞춰 주는."

팀장 : "그걸 바로 ES 에 얹지 말고, 더 단순한 걸로 한 번 연습해 봐요."

오픈이 : "단순한 거요?"

팀장 : "GitHub 에 올려 둔 `README.md` 가 바뀌면, 우리 서비스 로컬의 `README.md` 도 따라 바뀌게 해 봐요. 주기적으로 확인하는 쪽이랑, 받아서 반영하는 쪽이 따로 떠 있는 구조로."

오픈이는 턱을 괴고 생각했습니다. 왜 바로 ES 가 아닐까. 문득 알아챘습니다. 큐 구조를 처음 얹는데 ES 까지 한 번에 엮으면, 어디서 끊겼는지 모를 것 같았습니다. 파일 한 줄 바뀌는 건 눈으로 바로 보이니까, 큐만 따로 붙여 보라는 얘기였습니다.

키보드 위에 손을 올린 채 오픈이는 종이에 세 상자를 그렸습니다. 왼쪽 상자는 "관찰자", 가운데는 "큐", 오른쪽은 "반영자". 세 상자 사이에 화살표를 그었습니다. 화살표 위에 적힌 이름은 아직 없었습니다.

오픈이 : "관찰자는 GitHub 를 주기적으로 보는 거네요. 변경이 있으면 가운데로 넘기고, 오른쪽이 그걸 받아서 로컬 파일에 반영해요."

팀장 : "가운데 이름은 RabbitMQ 라고 붙여 두면 돼요."

오픈이는 종이에 `RabbitMQ` 라고 적었습니다. 세 상자 사이에 이름이 한 번에 채워졌습니다.

가운데를 뺄 수는 없을까?

순간 머리에 질문이 떠올랐습니다. 관찰자가 직접 반영자의 API 를 호출하면 되지 않나. 중간에 뭘 또 끼우나. 오픈이는 이 질문을 수첩에 적어 뒀습니다. 다음 절에서 직접 부딪혀 봐야 알 것 같았습니다.

9.2 한 줄로 엮었을 때, 끊어지면 같이 멈춘다

오픈이는 테스트 삼아 가장 단순한 구조를 그려 봤습니다. 관찰자가 GitHub 를 보고, 변경이 있으면 반영자의 REST API 를 직접 호출. 중간 큐 없이 두 서버를 한 줄로 엮는 구조입니다.

모니터에 두 터미널을 띄웠습니다. 왼쪽은 관찰자 로그, 오른쪽은 반영자 로그. 관찰자는 1 분에 한 번 GitHub 를 확인합니다. 변경이 감지되면 `POST http://반영자/update` 로 내용을 넘기고, 반영자가 파일을 쓴 뒤 `200 OK` 를 돌려주면 관찰자가 다음 루프로 넘어갑니다.

잠깐은 잘 됐습니다. GitHub 에 커밋 한 번을 누르자, 왼쪽 창에 `변경 감지` 가 찍히고, 오른쪽 창에 `파일 반영 완료` 가 바로 뒤따랐습니다.

이걸로도 충분한 거 아니야?

동료가 지나가다 모니터를 봤습니다.

동료 : "오픈이 님, 오른쪽 서버 내려 보면 어떻게 돼요?"

오픈이는 반영자 서버를 `Ctrl+C` 로 내렸습니다. 그리고 GitHub 에 커밋 한 번을 더 올렸습니다.

1 분 뒤, 왼쪽 창에 `변경 감지` 가 찍혔습니다. 그 뒤에 `Connection refused` 가 빨간색으로 길게 쏟아졌습니다. 관찰자의 스케줄러 스레드가 예외 때문에 멈췄습니다.

끊겼다.

동료 : "반영자가 잠깐만 내려가도 이벤트를 놓쳐요. 관찰자 쪽 루프도 예외로 엉키네요. 반영자를 올리고 나서 커밋 한 번 더 눌러야 다시 따라잡는 거예요?"

오픈이 : "그런 셈이네요."

동료 : "같은 파일을 두 번 올리면 두 번 반영돼요?"

오픈이는 손을 멈췄습니다. 지금 구조에서는 관찰자가 sha 를 기억하지 않으면, 같은 내용을 두 번 밀어 넣게 됩니다. 한 번 더 테스트해 보니 실제로 그랬습니다.

문제는 여기서 끝이 아니었습니다. 오픈이는 반영자 쪽에 `Thread.sleep(5000)` 을 잠깐 꽂아 봤습니다. 파일 쓰는 데 5 초가 걸린다고 가정하고 돌려 본 겁니다. 관찰자의 HTTP 호출이 5 초 동안 블로킹됐고, 그동안 관찰자의 스케줄러는 다음 폴링을 시작하지 못했습니다. 두 서버가 한 줄로 끈 묶여서, 한쪽이 느려지면 다른 쪽도 같이 느려지는 모양새였습니다.

관찰자와 반영자를 끊어 놔야 한다.

오픈이는 수첩을 펼쳐 세 가지를 적었습니다.

```
— 문제 정리 —

1. 장애 전파
   · 반영자가 내려가면 관찰자의 HTTP 호출이 실패
   · 예외가 스케줄러 스레드까지 올라와 루프가 엉킴

2. 이벤트 손실
   · 반영자 다운 중에 발생한 변경은 그대로 사라짐
   · 관찰자가 이벤트를 어디에도 보관하지 않음

3. 속도 결합
   · 반영자가 느려지면 관찰자도 같이 느려짐
   · HTTP 한 호출이 블로킹이기 때문
```

세 줄을 다 적고 나니, 팀장이 왜 "가운데에 하나 끼우라" 고 했는지 감이 왔습니다. 관찰자와 반영자 사이에 **메시지를 잠시 보관할 수 있는 자리** 가 있어야 합니다. 보관 자리가 있으면 반영자가 잠깐 내려가도 메시지가 거기 쌓여 있다가 복구 후 따라잡을 수 있고, 관찰자는 보관 자리에 던져만 두고 바로 다음 폴링을 돌릴 수 있습니다.

오픈이 : "그 보관 자리가 RabbitMQ 군요."

팀장 : "그렇죠."

9.3 생산자가 완성품을 벨트에 올려 두면, 소비자가 꺼내 간다

팀장이 자리에서 일어나 화이트보드 앞으로 갔습니다. 마커를 들고 길쭉한 선 하나를 가로로 그었습니다. 그 위에 상자 몇 개를 나란히 놓았습니다.

팀장 : "공장에서 생산자가 완성품을 컨베이어 벨트에 올려 두면, 소비자가 원할 때 집어가잖아요."

오픈이는 벨트 위의 상자를 쳐다봤습니다. 왼쪽 끝에 사람이 서서 상자를 벨트에 얹고 있었습니다. 벨트는 천천히 흐르고, 오른쪽 끝에서 다른 사람이 원하는 속도로 상자를 하나씩 집어 갔습니다. 두 사람이 서로 직접 얼굴을 맞대지 않습니다. 벨트가 둘을 분리합니다.

생산자가 벨트에 올려 두면, 소비자가 페이스대로 꺼내 간다.

팀장 : "만약 오른쪽 사람이 잠깐 자리를 비워도요?"

오픈이 : "상자가 벨트 위에 그대로 있죠."

팀장 : "돌아와서 쌓인 상자를 순서대로 집어 가면 돼요."

오픈이 : "왼쪽 사람은 오른쪽이 있든 없든 자기 페이스로 계속 올려 두면 되네요."

팀장 : "이 벨트를 프로그램으로 만든 게 RabbitMQ 에요."

오픈이는 수첩에 그림을 옮겨 그렸습니다. 그리고 각 부분에 이름을 붙였습니다. 왼쪽 사람 자리에는 **Producer**, 오른쪽 사람 자리에는 **Consumer**, 가운데 벨트 자리에는 **Broker** 라고 썼습니다. 벨트 위에는 상자가 여러 개 그려져 있었고, 그 상자 하나하나에 **Message** 라는 이름을 달았습니다.

그런데 팀장의 그림에는 벨트 앞에 **접수대** 가 하나 더 그려져 있었습니다. 생산자가 상자를 벨트에 바로 올리지 않고, 접수대에 먼저 내밀면 직원이 상자에 적힌 **라벨** 을 보고 어느 벨트로 보낼지 정하는 구조였습니다. 공장에 벨트가 여러 개 있으니까, 각 벨트에 맞춰 보내 줘야 한다는 얘기였습니다.

팀장 : "접수대가 Exchange 예요. 벨트는 Queue, 라벨은 Routing Key 예요."

오픈이 : "접수대에 '이 라벨이면 이 벨트' 라는 규칙이 걸려 있는 거네요."

팀장 : "그게 Binding 이에요."

오픈이는 우체국으로 비유를 바꿔 봤습니다. 택배 보내는 사람은 **Producer**, 택배에 붙인 운송장 코드가 **Routing Key** 입니다. 우체국 접수대가 **Exchange**, 접수대가 "이 운송장 코드는 A 지역 보관함으로" 하고 적어 둔 분류 규칙이 **Binding Key** 입니다. A 지역 보관함이 **Queue A**, 그 보관함에서 택배를 가져가는

기사가 **Consumer A** 입니다.

보내는 사람은 벨트에 올려 두는 게 아니라, 접수대에 내민다. 어느 벨트로 가는지 결정하는 건 접수대다.

팀장이 화이트보드에 숫자를 적었습니다.

팀장 : "세 가지만 맞추면 돼요. Exchange 이름, Queue 이름, Routing Key."

오픈이 : "세 이름이 양쪽에 같은 값으로 박혀 있어야 한다는 얘기죠."

팀장 : "그래서 실습 처음에 세 이름을 문서에 박아 두고 시작해요. 한 글자라도 어긋나면 메시지가 도착하지 않아요."

오픈이는 수첩에 세 이름을 적어 뒀습니다.

```
— 실습용 토폴로지 —

1. Exchange: github.events (Direct)
2. Queue: repo-updates
3. Routing Key: readme.changed
```

Direct Exchange 를 고른 이유도 팀장이 덧붙였습니다. Direct 는 "Routing Key 가 정확히 같을 때만 전달" 입니다. Topic 처럼 패턴을 허용하는 게 아니기 때문에, 이름이 한 글자라도 어긋나면 메시지가 아예 도착하지 않습니다. 실습에서는 이 엄격함이 오히려 디버깅을 쉽게 만듭니다. "어디가 안 맞는지" 를 바로 찾을 수 있기 때문입니다.

이제 개념은 잡혔습니다. 오픈이는 터미널을 열고 먼저 벨트부터 띄웠습니다.

9.4 Docker 로 RabbitMQ 한 컨테이너 띄우기

`rabbitmq-docker` 디렉토리로 이동해서 Docker Compose 를 실행합니다. 이미지는 `rabbitmq:3-management` 한 줄로 끝납니다. 관리 UI 까지 같이 들어 있어서 따로 깔 것이 없습니다.

`rabbitmq-docker/docker-compose.yml` 을 열고 아래 구성을 확인합니다.

실습 9-1 rabbitmq-docker/docker-compose.yml. RabbitMQ 컨테이너 구성

```
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    ports:
      - "5672:5672"    # AMQP 프로토콜
      - "15672:15672"  # 관리 UI
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
```

`rabbitmq:3-management` 는 RabbitMQ 브로커 본체와 웹 관리 UI 가 함께 묶인 이미지입니다. AMQP 는 Producer · Consumer 가 Spring AMQP 라이브러리로 접속할 때 쓰는 포트(`5672`) 이고, 관리 UI 는 브라우저에서 Exchange · Queue · Binding 을 눈으로 확인할 때 쓰는 포트(`15672`) 입니다. `guest/guest` 는 RabbitMQ 기본 계정으로, 로컬 실습에서만 쓰는 값이고 운영에서는 바꿔야 합니다.

터미널 실험 9-1 실행. RabbitMQ 컨테이너 기동

```
cd rabbitmq-docker
docker compose up -d
docker ps | grep rabbitmq
```

컨테이너가 올라왔는지 `docker ps` 로 확인하고, 브라우저에서 `http://localhost:15672` 로 접속합니다. 로그인 창이 뜨면 `guest/guest` 로 들어가면 됩니다.

그림 9-1. RabbitMQ 관리 UI 초기 상태입니다. 아직 Exchange · Queue 는 기본값만 보입니다.

Exchanges 탭을 열어 보면 `amq.direct` , `amq.topic` 같은 시스템 기본 Exchange 만 보입니다. 우리가 만들 `github.events` 는 아직 없습니다. Queues 탭도 비어 있습니다. Producer 가 한 번이라도 메시지를 발행해야, 앱에서 선언한 `github.events` Exchange 와 `repo-updates` Queue 가 나타납니다.

9.5 Producer · Consumer 가 같은 이름을 바라보게 고정한다

Exchange · Queue · Routing Key 세 이름을 양쪽 앱에 박아 두는 작업입니다. 한 글자라도 어긋나면 메시지는 발행돼도 도착하지 않습니다. 가장 흔한 실패 원인이기 때문에 이 절에서 세 이름을 문서에 고정합니다.

Producer 설정

`rabbitmq-producer/src/main/resources/application.properties` 를 열고 TODO 아래에 브로커 접속 정보와 토폴로지를 채웁니다.

실습 9-2 rabbitmq-producer/src/main/resources/application.properties. Producer 설정

```
# ===== RabbitMQ 브로커 연결 설정 =====
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

# ===== 메시징 토폴로지(양쪽에서 동일해야 함) =====
rabbit.exchange=github.events
rabbit.queue=repo-updates
rabbit.routing-key=readme.changed

# ===== GitHub 관찰 대상 =====
github.owner=본인계정
github.repo=config-readme
github.readme-path=README.md
github.poll-interval-ms=60000
```

`spring.rabbitmq.*` 네 줄은 AMQP 접속 정보입니다. `rabbit.exchange` · `rabbit.queue` · `rabbit.routing-key` 세 줄이 Producer · Consumer 가 반드시 공유해야 하는 토폴로지 이름입니다. `github.*` 네 줄은 관찰 대상 정보이고, `poll-interval-ms=60000` 은 1 분마다 GitHub 를 확인하겠다는 뜻입니다.

Consumer 설정

`rabbitmq-consumer/src/main/resources/application.properties` 에도 같은 토폴로지를 박습니다. Consumer 는 Exchange 이름 없이 Queue 이름만 필요하지만, 값을 맞춰 두면 헷갈리지 않습니다.

실습 9-3 rabbitmq-consumer/src/main/resources/application.properties. Consumer 설정

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

# Consumer 는 Producer 와 동일한 queue 이름을 사용해야 한다
rabbit.queue=repo-updates
```

Consumer 는 Exchange 에 직접 연결하지 않습니다. Consumer 가 구독하는 대상은 항상 Queue 한 개입니다. Exchange · Routing Key · Binding 은 Producer 쪽에서 생성·선언해 두면, Consumer 는 그 결과로 만들어진 Queue 이름만 알면 됩니다.

의존성 확인

두 앱 모두 `spring-boot-starter-amqp` 가 들어 있어야 합니다. 이 한 줄에 `RabbitTemplate`, `@RabbitListener`, `ConnectionFactory` 가 다 포함돼 있습니다.

실습 9-4 rabbitmq-producer/build.gradle. Producer · Consumer 공통 의존성

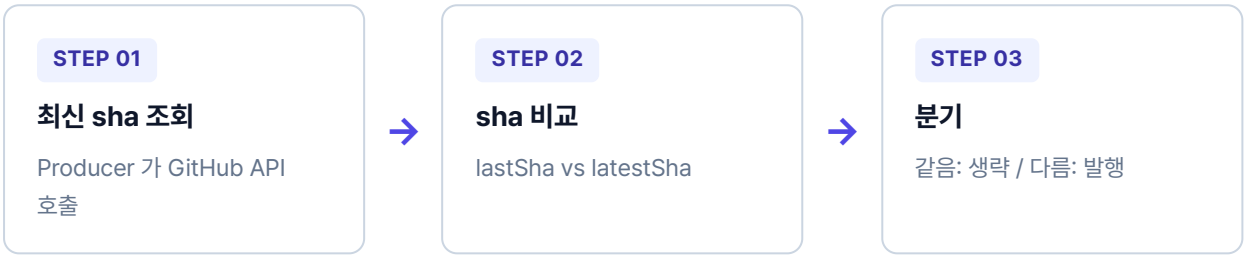
```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter'
    // 기타 의존성...
}
```

Consumer 의 `build.gradle` 도 같은 라인이 들어 있는지 한 번 더 확인합니다. 둘 중 한쪽만 추가하면, 한쪽은 정상 동작하더라도 다른 한쪽의 RabbitMQ 연동 코드가 아예 기동되지 않습니다.

9.6 무엇을 비교해서 변경을 알아낼까

RabbitMQ 는 띄워 놨고 두 앱이 같은 이름을 바라보게 했습니다. 이제 Producer 가 **GitHub 에서 무엇이 바뀐 걸 어떻게 알아내는지** 를 정해야 합니다.

그림 9-2. 폴링 변경 감지 흐름입니다



폴링 주기마다 최신 sha 만 한 번 조회하고, 값이 바뀌었을 때만 원문을 내려받아 메시지를 만듭니다.

비교 기준을 고르는 것이 첫 결정입니다. 후보는 네 가지입니다.

비교 기준	장점	단점	추천 상황
sha (커밋/파일 버전 식별자)	가장 명확하고 로그에 남기기 쉬움	Git 상태 기준이라 구현에 따라 커밋 단위가 됨	Git 으로 관리되는 파일 (README.md) 기본값
ETag (HTTP 캐시 검증자)	조건부 요청으로 네트워크 비용↓	서버/엔드포인트마다 동작이 다를 수 있음	HTTP 기반으로 가볍게 감지하고 싶을 때
Last-Modified (수정 시각)	이해·구현이 간단	시간 정밀도가 애매한 경우가 있음	대략적인 변경 감지가 허용될 때
Content Hash (내용 해시)	내용이 실제로 바뀌었는지 최종 보장	내용을 내려받아야 계산 가능(비용 큼)	최종 검증용(sha 로 1차 감지 후 2차 확인)

README.md 처럼 Git 으로 관리되는 파일은 **sha 비교** 가 기본입니다. 커밋이 바뀌면 sha 도 바뀌기 때문에 "어느 커밋까지 반영했는가" 를 하나의 값으로 기록할 수 있습니다. 폴링 주기마다 네트워크를 타는 건 커밋 목록 조회 한 번이고, 변경이 있을 때만 원문을 내려받기 때문에 호출 비용도 크지 않습니다.

주기 선택은 단순한 숫자 문제가 아닙니다. 주기가 짧을수록 변경 감지는 빨라지지만, GitHub API 호출이 많아져 **레이트리밋**(호출 제한) 에 걸리기 쉽습니다. 여러 파일·여러 저장소를 동시에 폴링하면 "주기 × 대상 개수" 만큼 호출이 늘어나므로 수치는 더 커집니다. 실습에서는 **60 초** 로 잡아 동작 확인을 빠르게 하고, 운영에서는 1~5 분 범위에서 시작해 변경 빈도·호출 한도를 보고 조정하는 편이 좋습니다.

중복 발행 방지 는 폴링 구조에서 반드시 필요합니다. 폴링은 같은 sha 를 여러 번 읽을 수 있기 때문에, 이전에 확인한 `lastSha` 를 저장해 두고 최신 `latestSha` 와 비교해야 합니다. 같으면 발행을 생략하고, 다를 때만 메시지를 만들고 `lastSha` 를 갱신합니다. 실습에서는 단순히 인스턴스 메모리 필드에 `lastSha` 를 둡니다. 운영에서는 재시작·다중 인스턴스에서도 마지막 상태가 유지되도록 Redis 나 DB 같은 외부 저장소에 보관하는 편이 안전합니다.

9.7 관찰 서버. 폴링에서 발행까지

Producer 는 세 조각으로 나뉩니다. GitHub 에서 최신 sha 와 원문을 가져오는 `GitHubClient`, 주기마다 돌면서 변경을 판단하는 `PollingScheduler`, 그리고 변경이 감지됐을 때 RabbitMQ 로 메시지를 보내는 `RabbitProducer` 입니다. `RabbitDTO` 는 세 조각이 공유하는 메시지 계약입니다.

9.7.1 GitHub 에서 최신 sha 와 원문 가져오기

`rabbitmq-producer/src/main/java/com/rabbitmq/producer/github/GitHubClient.java` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 9-5 rabbitmq-producer/src/main/java/com/rabbitmq/producer/github/GitHubClient.java. 최신 sha 조회

```
public String fetchLatestSha() {
    // 1. 커밋 목록 URL 만들기. per_page=1 로 최신 1 건만
    String url = String.format(
        "https://api.github.com/repos/%s/%s/commits?path=%s&per_page=1",
        owner, repo, readmePath
    );

    // 2. REST 호출. 응답은 커밋 객체 리스트
    ResponseEntity<List<Map<String, Object>>> response =
        restTemplate.exchange(url, HttpMethod.GET, new HttpEntity<>(headers),
            new ParameterizedTypeReference<>() {});
```

```
    // 3. 첫 번째 커밋의 sha 꺼내기
    List<Map<String, Object>> commits = response.getBody();
    if (commits == null || commits.isEmpty()) return null;
    Object sha = commits.get(0).get("sha");
    return sha != null ? sha.toString() : null;
}
```

`fetchLatestSha()` 는 `owner` · `repo` · `readmePath` 를 조합해 GitHub Commits API URL 을 만듭니다. `per_page=1` 은 "이 파일 경로에 영향을 준 최신 커밋 1 건만 달라" 는 뜻이고, 응답은 커밋 객체 배열이기 때문에 첫 번째 원소의 `sha` 필드를 꺼내 문자열로 반환합니다. 폴링 루프는 이 값 하나로 변경 여부를 판단합니다.

같은 파일에 원문 조회 메서드도 함께 작성합니다.

실습 9-6 rabbitmq-producer/src/main/java/com/rabbitmq/producer/github/GitHubClient.java. README 원문 조회

```
public String fetchReadmeContent() {
    // 1. raw.githubusercontent.com 주소 조합
    String rawUrl = String.format(
        "https://raw.githubusercontent.com/%s/%s/main/%s",
        owner, repo, readmePath
    );
    // 2. 문자열 그대로 내려받기
    return restTemplate.getForObject(rawUrl, String.class);
}

public String getRepoFullName() {
    return owner + "/" + repo;
}
```

`fetchReadmeContent()` 는 `raw.githubusercontent.com` 주소로 원문을 내려받습니다. 이 엔드포인트는 커밋 메타가 아니라 파일의 생 텍스트를 돌려주기 때문에, 그대로 메시지의 `content` 필드에 넣을 수 있습니다. 기본 브랜치가 `main` 이 아닌 경우에는 URL 의 `main` 자리를 실제 브랜치명으로 바꿔야 합니다. `getRepoFullName()` 은 `owner/repo` 형태의 식별자를 조립해 메시지의 `repo` 필드로 넣기 위해 제공합니다.

메서드	호출 시점	반환 값
`fetchLatestSha()`	폴링 주기마다 1 회	최신 커밋의 sha 문자열
`fetchReadmeContent()`	변경이 감지됐을 때만	README.md 원문 문자열

`getRepoFullName()`	메시지 빌드 시	`owner/repo` 식별자

9.7.2 폴링 스케줄러. 변경 판단과 발행 호출

`rabbitmq-producer/src/main/java/com/rabbitmq/producer/scheduler/PollingScheduler.java` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 9-7 rabbitmq-producer/src/main/java/com/rabbitmq/producer/scheduler/PollingScheduler.java. 폴링 루프

```
private String lastSha = null;

@Scheduled(fixedRateString = "${github.poll-interval-ms}")
public void checkForReadmeChange() {
    // 1. 최신 sha 조회
    String latestSha = gitHubClient.fetchLatestSha();
    if (latestSha == null) return;

    // 2. 이전 sha 와 비교. 같으면 생략
    if (latestSha.equals(lastSha)) return;

    // 3. 다르면 lastSha 갱신 후 원문 조회
    lastSha = latestSha;
    String content = gitHubClient.fetchReadmeContent();

    // 4. 메시지 조립
    RabbitDTO message = RabbitDTO.builder()
        .repo(gitHubClient.getRepoFullName())
        .sha(latestSha)
        .content(content)
        .timestamp(LocalDateTime.now())
        .build();

    // 5. RabbitMQ 로 발행
    rabbitProducer.send(message);
}
```

`@Scheduled(fixedRateString = "${github.poll-interval-ms}")` 는 `application.properties` 의 값을 읽어 주기를 결정합니다. 실습에서는 60000(60 초) 입니다. `lastSha` 는 인스턴스 메모리에만 존재하는 값이므로 앱을 재시작하면 `null` 로 되돌아갑니다. 그래서 서버가 막 뜬 직후 첫 폴링에서는 "변경 감지" 로 판단하고 최초 커밋을 한 번 발행합니다. 그 이후로는 GitHub 에 새 커밋이 올라올 때만 3~5 단계가 실행됩니다.

매개변수·필드	값	출처
`lastSha`	인스턴스 메모리(기본 `null`)	`PollingScheduler` 내부 필드
`latestSha`	GitHub 응답	`gitHubClient.fetchLatestSha()`
`poll-interval-ms`	60000	`application.properties`
`gitHubClient`	`GitHubClient` 빈	생성자 주입
`rabbitProducer`	`RabbitProducer` 빈	생성자 주입

9.3 에서 팀장이 말한 "벨트에 올려 두면 된다" 는 한 줄이 이 메서드 안의 `rabbitProducer.send(message)` 한 줄입니다. 관찰자가 오른쪽 반영자의 상태를 신경 쓰지 않고, 메시지를 벨트(Queue) 에 얹는 동작 한 번으로 자기 할 일을 마칩니다.

9.7.3 메시지 계약과 발행

`rabbitmq-producer/src/main/java/com/rabbitmq/producer/dto/RabbitDTO.java` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 9-8 rabbitmq-producer/src/main/java/com/rabbitmq/producer/dto/RabbitDTO.java. 메시지 계약

```
@Getter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class RabbitDTO {
    private String repo;
    private String sha;
    private String content;
    private LocalDateTime timestamp;
}
```

`RabbitDTO` 는 Producer 와 Consumer 가 **똑같이 공유해야 하는 메시지 계약** 입니다. `repo` 에 변경 대상 저장소 식별자, `sha` 에 변경 기준 커밋 해시, `content` 에 README 원문, `timestamp` 에 발행 시각을 담습니다. 필드명이 한 글자라도 어긋나면 Consumer 쪽에서 JSON → DTO 역직렬화가 실패합니다. 그래서 실습에서는 Producer 의 DTO 파일과 Consumer 의 DTO 파일을 같은 구조로 들고 있습니다.

`rabbitmq-producer/src/main/java/com/rabbitmq/producer/rabbit/RabbitProducer.java` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 9-9 rabbitmq-producer/src/main/java/com/rabbitmq/producer/rabbit/RabbitProducer.java. 발행

```
public void send(RabbitDTO message) {
    // 1. Exchange 이름과 Routing Key 로 메시지 발행
    rabbitTemplate.convertAndSend(exchange, routingKey, message);
    // 2. 로그로 어떤 payload 가 어디로 나갔는지 남김
    log.info("[RabbitMQ] 메시지 발행 → {}", message);
}
```

`rabbitTemplate.convertAndSend(exchange, routingKey, message)` 한 줄이 이 챕터 전체의 핵심입니다. 첫 번째 인자는 Exchange 이름(`github.events`), 두 번째 인자는 Routing Key(`readme.changed`), 세 번째 인자는 실제 payload 입니다. Spring AMQP 가 기본으로 등록된 `MessageConverter` 를 사용해 `RabbitDTO` 를 JSON 으로 직렬화한 뒤 AMQP 프로토콜로 브로커에 넘깁니다. Exchange 와 Queue 가 아직 브로커에 없다면 Spring 이 앱 기동 시 자동으로 선언해 줍니다.

9.7.4 Producer 한 번 실행해 보기

Consumer 는 아직 만들기 전이지만, Producer 쪽만 먼저 띄워서 메시지가 RabbitMQ 의 Queue 에 쌓이는지 확인해 보겠습니다.

터미널 실험 9-2 실행. Producer 단독 실행

```
cd rabbitmq-producer
./gradlew bootRun
```

서버가 뜨면 최초 폴링에서 `lastSha = null` 이므로 현재 커밋 sha 로 바로 "변경 감지" 분기를 타게 됩니다. 로그에 순서대로 세 줄이 찍히는지 확인합니다.

그림 9-3. Producer 첫 실행 시 감지 → 커넥션 → 발행 로그가 순서대로 출력됩니다.

이 상태에서 `http://localhost:15672` 의 Exchanges 탭을 새로고침하면 `github.events` 가 목록에 생겨 있고, Queues 탭의 `repo-updates` 에는 "Ready: 1" 이 찍혀 있습니다. 벨트 위에 상자 하나가 올라가 있는 상태입니다. Consumer 가 아직 가져가지 않았기 때문에 그대로 쌓여 있습니다.

그림 9-4. Consumer 없이도 `repo-updates` 큐에 메시지가 쌓여 있습니다.

9.8 수신 서버. 큐를 구독하고 파일에 반영한다

이제 벨트에서 상자를 집어 갈 사람을 만듭니다. Consumer 는 두 조각입니다. Queue 를 Spring 빈으로 선언하고 JSON 컨버터를 등록하는 `RabbitConfig` , 그리고 메시지를 받아 파일에 반영하는 `RabbitConsumer` 입니다.

9.8.1 큐 빈 등록과 JSON 컨버터

`rabbitmq-consumer/src/main/java/com/rabbitmq/consumer/config/RabbitConfig.java` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 9-10 rabbitmq-consumer/src/main/java/com/rabbitmq/consumer/config/RabbitConfig.java. Queue · Converter 빈

```
@Configuration
public class RabbitConfig {

    @Value("${rabbit.queue}")
    private String queueName;

    // 1. Queue 를 Spring 빈으로 선언. true 는 durable (재시작 후에도 유지)
    @Bean
    public Queue queue() {
        return new Queue(queueName, true);
    }

    // 2. JSON ↔ DTO 컨버터. Producer 와 포맷을 맞추기 위해 필수
    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}
```

`Queue` 빈을 등록하면 Consumer 앱이 기동될 때 RabbitMQ 에 `repo-updates` 큐가 존재하지 않으면 자동으로 만들고, 이미 있으면 연결만 합니다. `true` 인자는 **durable**(내구성) 설정입니다. 브로커가 재시작돼도 큐가 유지되고, 큐에 쌓인 메시지도 살아남습니다.

`Jackson2JsonMessageConverter` 빈은 이 실습에서 가장 자주 틀리는 자리입니다. Producer 가 `convertAndSend` 로 객체를 보낼 때 Spring AMQP 는 등록된 `MessageConverter` 를 사용해 객체를 JSON 바이트로 바꿉니다. Consumer 쪽에 같은 컨버터가 등록돼 있지 않으면, 들어온 바이트를 기본 자바 직렬화로 읽으려다 실패합니다. 결과는 `MessageConversionException` 과 함께 Consumer 로그가 뻘겋게 물드는 모습입니다. 두 앱이 같은 컨버터를 쓰도록 맞춰 두는 것이 이 한 빈의 역할입니다.

9.8.2 리스너. 큐 구독과 파일 반영

`rabbitmq-consumer/src/main/java/com/rabbitmq/consumer/dto/RabbitDTO.java` 는 Producer 쪽 DTO 와 동일한 필드로 둡니다. 이 파일은 앞 절의 실습 9-8 과 구조가 같기 때문에 복사만 하면 됩니다.

`rabbitmq-consumer/src/main/java/com/rabbitmq/consumer/listener/RabbitConsumer.java` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 9-11 rabbitmq-consumer/src/main/java/com/rabbitmq/consumer/listener/RabbitConsumer.java. 메시지 수신

```
private static final String README_PATH = "./README.md";

@RabbitListener(queues = "${rabbit.queue}")
public void receive(RabbitDTO message) {
    // 1. 수신 로그
    System.out.println("=== 메시지 수신 ===");
    System.out.println("Repository: " + message.getRepo());
    System.out.println("SHA: " + message.getSha());
    System.out.println("타임스탬프: " + message.getTimestamp());

    // 2. 파일 반영
    try {
        patchFile(message.getContent());
    } catch (IOException e) {
        System.err.println("파일 업데이트 중 오류 발생: " + e.getMessage());
    }
}
```

`@RabbitListener(queues = "${rabbit.queue}")` 는 "`rabbit.queue` 프로퍼티에 적힌 큐를 구독해 달라" 는 선언입니다. Spring AMQP 가 앱 기동 시 해당 큐에 대한 컨슈머 채널을 연결하고, 브로커가 메시지를 푸시해 주면 `receive(RabbitDTO)` 메서드가 자동 호출됩니다. 인자 타입이 `RabbitDTO` 이기 때문에 Spring 은 9.8.1 에서 등록한 `Jackson2JsonMessageConverter` 로 JSON 을 역직렬화해 객체를 만들어 넣어 줍니다. Consumer 코드에서는 AMQP 프로토콜·바이트 해석을 직접 다룰 필요가 없습니다.

같은 파일에 `patchFile()` · `writeContent()` 를 이어서 작성합니다.

실습 9-12 rabbitmq-consumer/src/main/java/com/rabbitmq/consumer/listener/RabbitConsumer.java. 백업 + 덮어쓰기

```
private void patchFile(String newContent) throws IOException {
    File file = new File(README_PATH);

    // 1. 파일이 없으면 새로 만들고 끝
    if (!file.exists()) {
        writeContent(file, newContent);
        return;
    }

    // 2. 기존 내용과 같으면 생략
    String oldContent = Files.readString(file.toPath(), StandardCharsets.UTF_8);
    if (Objects.equals(oldContent.trim(), newContent.trim())) return;

    // 3. 백업 폴더 준비
    File backupDir = new File(file.getParent() == null ? "." : file.getParent(), "back
ups");
    if (!backupDir.exists()) backupDir.mkdirs();

    // 4. 타임스탬프 이름으로 백업 저장
    String backupFileName = String.format("readme_backup_%d.md", System.currentTimeMil
lis());
    File backupFile = new File(backupDir, backupFileName);
    Files.copy(file.toPath(), backupFile.toPath(), StandardCopyOption.REPLACE_EXISTIN
G);

    // 5. 새 내용으로 덮어쓰기
    writeContent(file, newContent);
}

private void writeContent(File file, String content) throws IOException {
    try (Writer writer = new OutputStreamWriter(
            new FileOutputStream(file, false), StandardCharsets.UTF_8)) {
        writer.write(content);
```

```
    }
}
```

`patchFile()` 은 네 가지 상황을 한 번에 처리합니다. 파일이 없으면 새로 만들고, 있으면 기존 내용과 비교해 같으면 생략하며, 다르면 백업을 만든 뒤 덮어씁니다. 백업은 같은 디렉토리의 `backups/` 폴더 아래에 `readme_backup_<timestamp>.md` 이름으로 저장됩니다. 운영에서 실수로 원본이 엎어졌을 때 이전 버전을 찾아 낼 수 있는 최소한의 안전망입니다.

`writeContent()` 는 `try-with-resources` 로 `Writer` 를 열어 UTF-8 로 파일을 덮어씁니다. 한국어가 깨지지 않게 하는 핵심은 `StandardCharsets.UTF_8` 한 인자입니다. 그리고 `FileOutputStream(file, false)` 의 `false` 는 append 가 아니라 **truncate** 모드, 즉 파일을 비우고 새로 쓰겠다는 뜻입니다.

9.9 통합 테스트. 커밋 한 번으로 세 프로세스가 움직인다

이제 세 프로세스를 순서대로 띄우고, GitHub 에 커밋 하나를 눌러 전체 파이프라인을 확인합니다.

메시지 기반 시스템은 **수신자가 먼저 떠 있어야** 로그가 깔끔합니다. Consumer 를 먼저 띄워야 Producer 가 발행하는 순간 바로 수신 로그가 이어집니다. 반대로 올리면 Producer 가 먼저 발행한 메시지가 Queue 에 쌓여 있다가 Consumer 가 뜬 직후 한 번에 소비됩니다. 기능상 문제는 없지만, 실습에서는 흐름을 눈으로 따라가기 어렵습니다.

터미널 실험 9-3 실행. 세 프로세스 순서대로 기동

```
# 1. RabbitMQ (터미널 1)
cd rabbitmq-docker && docker compose up -d

# 2. Consumer (터미널 2)
cd rabbitmq-consumer && ./gradlew bootRun

# 3. Producer (터미널 3)
cd rabbitmq-producer && ./gradlew bootRun
```

세 프로세스가 올라오면 Producer 쪽에서 최초 1 회 "변경 감지" 로그가 찍히고, Consumer 쪽에서 "메시지 수신" 로그가 이어집니다. Consumer 앱의 실행 디렉토리에 `README.md` 가 생성되고, GitHub 의 최초 커밋 내용이 그대로 채워집니다.

GitHub 에서 README 를 수정한다

브라우저에서 `config-readme` 레포로 가서 `README.md` 를 편집하고 커밋합니다. 예를 들어 `# Test Update` 라는 한 줄을 추가해 커밋을 누릅니다. GitHub 쪽에서 커밋이 생성된 뒤, 다음 폴링 주기(최대 60 초) 안에 Producer 로그가 다시 찍히기 시작합니다.

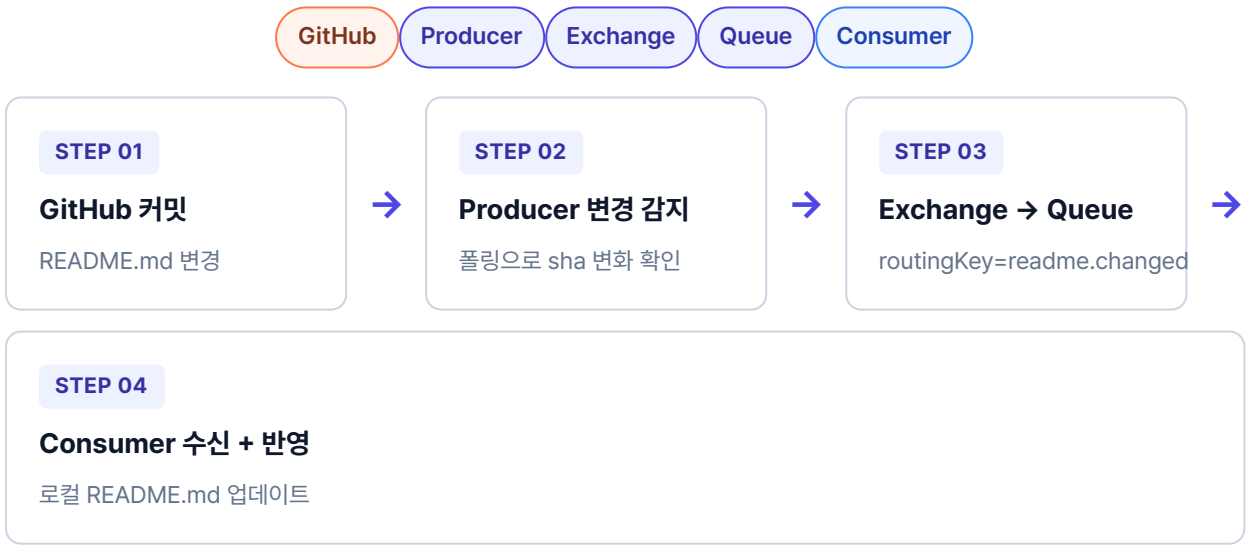
그림 9-5. Producer 가 새 커밋을 감지하고 메시지를 발행하는 순간입니다.

그 직후 Consumer 터미널이 깜박입니다.

그림 9-6. Consumer 가 같은 메시지를 받아 로컬 `README.md` 에 반영합니다.

Consumer 앱 디렉토리의 `README.md` 를 열어 보면 GitHub 에서 추가한 `# Test Update` 한 줄이 그대로 박혀 있습니다. `backups/` 폴더 안에는 이전 내용이 담긴 `readme_backup_<timestamp>.md` 가 하나 생성돼 있습니다.

그림 9-7. 커밋 → 발행 → 수신 → 반영의 한 사이클입니다



네 단계가 한 사이클입니다. 세 프로세스가 서로의 상태를 모른 채 제 페이스로 움직입니다.

벨트가 돌았다. 오픈이는 세 터미널의 로그를 위에서 아래로 한 번 훑어봤습니다. 커밋을 누른 것은 브라우저 하나였는데, 세 창에서 동시에 줄이 올라갔습니다.

동료가 뒤에서 모니터를 봤습니다.

동료 : "이제 Consumer 쪽을 잠깐 내려 볼까요?"

오픈이는 Consumer 를 `Ctrl+C` 로 내리고, GitHub 에 커밋 한 번을 더 올렸습니다. Producer 로그에는 1 분 뒤 "변경 감지" 가 찍혔고, 새로운 메시지가 브로커의 `repo-updates` 큐에 들어갔습니다. 관리 UI 의 Queues 탭을 보면 Ready 카운트가 1 로 올라 있었습니다. Consumer 가 없으니 아무도 집어 가지 않고 쌓여 있는 모습이었습니다.

다시 Consumer 를 띄우자, 쌓여 있던 메시지가 즉시 소비되면서 로컬 파일이 한 번에 업데이트됐습니다.

동료 : "반영자가 잠깐 내려가도 메시지가 안 사라지네요."

오픈이 : "벨트 위에 그대로 있다가, 돌아오면 집어 가는 거예요."

9.2 에서 적어 둔 세 문제. 장애 전파·이벤트 손실·속도 결합. 세 개가 한 번에 풀렸습니다. RabbitMQ 가 사이에 들어오자 관찰자와 반영자가 서로의 상태를 보지 않게 됐습니다.

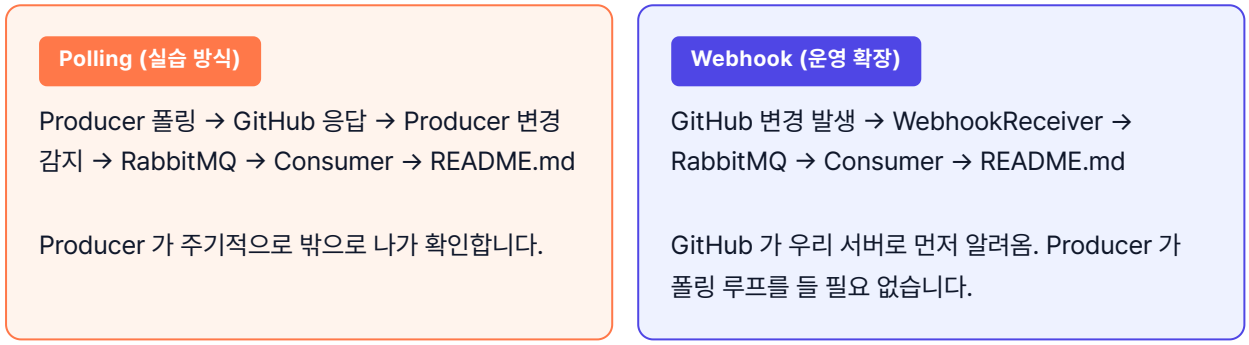
RabbitMQ 실습에서 생각해 볼 것들

- **내구성과 메시지 영속**: `new Queue(queueName, true)` 의 `true` 는 큐 메타의 durable 입니다. 메시지 자체가 디스크로 내려가려면 Producer 쪽에서도 persistent 메시지로 발행해야 합니다. Spring AMQP 는 기본으로 persistent 로 보내기 때문에 이 실습에서는 자연스럽게 유지됩니다
- **소비 실패 처리**: `@RabbitListener` 메서드 안에서 예외가 터지면 기본 동작은 재시도 후 Dead Letter 로 보내는 것이 아니라, 같은 메시지가 다시 큐로 돌아오는 모양이 됩니다. 운영에서는 DLX(Dead Letter Exchange) 와 재시도 정책을 반드시 함께 설계합니다
- **멱등성(Idempotency)**: 같은 메시지가 두 번 도착해도 최종 상태가 같도록 Consumer 를 설계해야 합니다. 이 실습에서는 `patchFile()` 이 기존 내용과 같으면 생략하기 때문에 이미 멱등에 가깝습니다
- **Prefetch 와 수동 Ack**: 한 Consumer 가 한 번에 집어갈 메시지 수(`prefetch`), 수동 Ack/Nack 설정은 처리 지연·실패 시 재처리에 큰 영향을 줍니다. 운영에서는 기본값 대신 워크로드에 맞춰 조정하는 자리입니다
- **메시지 크기**: 실습에서는 `content` 필드에 README 원문을 통째로 넣었지만, 운영에서는 파일이 커지면 메시지 크기가 문제가 됩니다. URL · sha 만 싣고 Consumer 가 필요 시 원문을 다시 조회하는 구조가 더 자주 쓰입니다

9.10 Polling 을 Webhook 으로 바꿔 본다

실습은 Polling 으로 만들었지만, 운영에서는 Webhook 방식이 더 자주 쓰입니다. Polling 은 "우리가 주기적으로 나가서 확인" 하는 방식이고, Webhook 은 "변경이 생기면 GitHub 가 먼저 우리 서버로 알려주는" 방식입니다. Webhook 을 쓰면 "변경이 없는데 확인만 하는" 호출이 사라집니다.

그림 9-8. Polling 과 Webhook 구조 비교입니다



구조에서 달라지는 자리는 Producer 입니다. Polling 에서는 `@Scheduled` 로 주기마다 GitHub 에 확인 요청을 보냈지만, Webhook 에서는 GitHub 가 먼저 우리 서버의 특정 URL 로 HTTP POST 를 쏴 줍니다. 그 요청을 받는 **WebhookReceiver** 한 컨트롤러가 Producer 역할을 대신합니다. 받은 페이로드에서 README 변경 여부를 판단해 RabbitMQ 로 발행하면, 뒷부분(Exchange · Queue · Consumer) 은 이 실습과 동일합니다.

Webhook 을 쓰려면 **공개 주소** 가 필요합니다. `localhost:8080` 은 내 컴퓨터 안에서만 보이기 때문에 GitHub 가 접근할 수 없습니다. 운영에서는 공인 IP · 도메인 · 로드밸런서 같은 외부 진입점을 두고, 개발 중에는 터널링 도구로 임시 주소를 만들어 테스트합니다. 그리고 요청 본문이 정말 GitHub 가 보낸 것인지 확인하기 위해 Webhook Secret 으로 서명 검증을 해야 합니다. 그렇지 않으면 누구나 `/webhook` 엔드포인트로 가짜 이벤트를 쏠 수 있습니다.

정리하면 Polling 은 "우리가 밖으로 나가는 모델" 이고 Webhook 은 "밖에서 우리 쪽으로 들어오는 모델" 입니다. 로컬 실습은 Polling 이 편하고, 운영은 Webhook 이 비용·지연 양쪽에서 유리합니다. 어느 쪽을 골라도 **RabbitMQ 뒷단(Exchange · Queue · Consumer) 은 그대로 재사용** 할 수 있습니다. 챕터 초반에 팀장이 "가운데에 하나 끼우라" 고 했던 이유가 여기에도 걸립니다. 가운데가 단단하면 앞단의 구현 방식이 바뀌어도 뒷단은 흔들리지 않습니다.

9.11 아홉 기둥을 돌아본다

됐다. 오픈이는 세 터미널을 한 번에 봤습니다. Producer · Consumer · RabbitMQ 관리 UI. GitHub 브라우저 창까지 합치면 네 장의 풍경이었습니다. 뒤로 기대 앉아 천장을 봤더니, 그동안 오픈이의 서비스에 하나씩 얹혀 온 기둥들이 줄지어 지나갔습니다.

처음은 챕터 1, **Docker** 였습니다. "내 로컬에선 되는데" 를 없애려고 이미지 한 장을 만들었습니다. 컨테이너 하나가 오픈이의 PC 와 팀의 리눅스 서버 사이의 번역기가 됐고, 그날부터 "A 는 되는데 B 는 안 됨" 이라는 문장이 사라졌습니다.

챕터 2·3 에서는 **카카오 로그인**과 **Redis 세션** 이 붙었습니다. 비밀번호를 직접 들고 있지 않고 통행증(OAuth) 으로 인증을 위임했고, 서버가 한 대에서 여러 대로 늘어나자 세션을 바깥(Redis) 에 꺼내 모든 서버가 같은 지갑을 들여다보도록 만들었습니다.

챕터 4·5 는 **파일** 이었습니다. 처음에는 이미지를 Base64 로 밀어 넣어 봤다가, 트래픽이 커지자 Presigned URL · S3 · Lambda 로 서버의 어깨에서 파일 처리를 덜어 냈습니다. 서버는 주소만 내주고, 진짜 파일은 클라이언트와 S3 가 직접 주고받는 구조였습니다.

챕터 6·7 은 **실시간과 스트리밍** 이었습니다. Polling · SSE · WebSocket 세 방식을 상황에 따라 골라 쓰는 법을 익혔고, 대용량 영상은 HLS 로 쪼개어 필름 릴처럼 조각 단위로 스트리밍했습니다.

챕터 8 은 **검색** 이었습니다. `LIKE '%강의%'` 의 5 초를 역색인 카드 한 장으로 ms 까지 끌어내렸고, 그 끝에서 "두 저장소를 쓴다" 는 구조가 부를 문제를 살짝 건드렸습니다. 이 챕터 9 가 그 문제의 해답 쪽 방향이었습니다.

그리고 챕터 9, 지금 이 자리에서 **이벤트 동기화** 를 얹었습니다. 관찰자와 반영자를 벨트 하나로 끊어 놓자, 한쪽이 내려가도 다른 쪽이 멈추지 않고, 메시지가 사라지지 않으며, 서로의 속도를 방해하지 않았습니다.

아홉 장을 지나오면서 서비스에는 여섯 기둥이 세워졌습니다. **인증**(OAuth) · **세션·캐시**(Redis) · **파일**(S3) · **실시간**(SSE · WebSocket · HLS) · **검색**(Elasticsearch) · **이벤트**(RabbitMQ). 처음 챕터 1 의 "내 로컬에선 되는데" 가 "여러 대의 서버가, 여러 종류의 저장소를 들고, 여러 방향에서 들어오는 요청을 각자의 속도로 받아내는" 그림까지 와 있었습니다.

여기까지 왔다.

팀장이 오픈이 자리 옆으로 지나가다 화면을 힐끗 보고 말했습니다.

팀장 : "여섯 기둥을 다 얹은 셈이네요."

오픈이 : "네. 기둥마다 처음엔 왜 필요한지도 몰랐어요."

팀장 : "문제를 만나기 전엔 그래요. 한 번씩 부딪혀 보면 기둥이 왜 거기에 박혀 있는지 몸에 남아요."

동료가 지나가며 끼어들었습니다.

동료 : "근데 오픈이 님, 그 여섯 개 말고 아직 안 건드린 것들도 많잖아요."

오픈이 : "그건 다음 서비스에서요."

오픈이는 수첩을 덮었습니다. 책상에는 커피잔이 절반쯤 비어 있고, 모니터에는 세 터미널이 여전히 제 페이스로 움직이고 있었습니다. 첫날 팀장이 "내 로컬에선 되는데요?" 라고 물었던 오픈이와, 지금 RabbitMQ 의 큐 앞에 앉아 있는 오픈이는 같은 사람이지만 다른 사람이었습니다.

용어 정리

이야기 속 표현	진짜 용어	정식 정의
가운데 벨트 · 우체국 시스템	메시지 브로커 (Message Broker)	Producer 와 Consumer 사이에서 메시지를 받아 분류·보관·전달하는 중계 서버. RabbitMQ · Kafka · ActiveMQ 등이 해당. 송수신 측이 서로 직접 연결되지 않게 분리하는 역할
상자를 올리는 사람	Producer	메시지를 만들어 Exchange 로 발행하는 주체. 메시지에 Routing Key 를 붙여 보내며, 수신자의 상태를 직접 확인하지 않음
상자를 집는 사람	Consumer	특정 Queue 를 구독하여 메시지를 소비하는 주체. 자신의 처리 속도에 맞춰 한 건씩 또는 `prefetch` 단위로 가져감

우체국 접수대	Exchange	Producer 가 발행한 메시지를 받아, 설정된 Binding 규칙에 따라 하나 이상의 Queue 로 라우팅하는 엔터티. 타입으로 Direct · Topic · Fanout · Headers 가 있음
보관함 대기열	Queue	메시지를 수신자가 가져가기 전까지 잠시 보관하는 버퍼. durable · auto-delete · exclusive 옵션에 따라 수명과 공유 정책이 달라짐
운송장 코드	Routing Key	Producer 가 메시지에 붙여서 Exchange 에 넘기는 라우팅 라벨 문자열. `order.created` 같은 점 구분 이름이 관례
접수대의 분류 규칙	Binding (Binding Key)	Exchange 와 Queue 사이의 연결. Routing Key 를 어떤 Queue 로 보낼지 정하는 규칙. Direct 는 정확 일치, Topic 은 `*` · `#` 패턴 매칭
접수대가 정확 일치만 통과시킴	Direct Exchange	Routing Key 가 Binding Key 와 **정확히 같을 때** 만 해당 Queue 로 전달하는 Exchange 타입. 실습·디버깅에 유리
상자에 적힌 내용	payload	메시지 본문. AMQP 프로토콜에서는 바이트 배열이며, Spring AMQP 가 `MessageConverter` 로 객체 ↔ 바이트 변환
상자 <-> 객체 변환기	`Jackson2JsonMessageConverter`	Spring AMQP 의 JSON 기반 `MessageConverter` 구현체. Producer 에서는 객체 → JSON, Consumer 에서는 JSON → 객체 변환
상자 발행기	`RabbitTemplate.convertAndSend(exchange, routingKey, payload)`	Spring AMQP 의 발행 API. 내부에서 `MessageConverter` 를 거쳐 AMQP 프로토콜로 브로커에 전달

상자 청취기	`@RabbitListener(queues = "...")`	특정 Queue 를 구독하는 메서드를 선언하는 어노테이션. 메시지가 들어오면 자동 호출. 메서드 파라미터 타입에 맞춰 역직렬화
주기 확인	Polling	수신 측이 주기적으로 원본에 "변경 있느냐" 를 물어 상태를 가져오는 방식. 간단하지만 변경 없는 호출 비용이 발생
먼저 알려줌	Webhook	변경이 생길 때 원본 서버가 수신자에게 HTTP 요청으로 직접 알리는 방식. 호출 비용이 낮지만 수신자가 공개 주소와 서명 검증을 갖춰야 함
한 번 본 것 기록	`lastSha`	Producer 가 이전 폴링에서 관찰한 최신 sha 를 보관해 두는 값. 동일 변경을 여러 번 발행하지 않도록 `latestSha` 와 비교
호출 제한	레이트리밋 (Rate Limit)	API 서버가 클라이언트당 단위 시간 내 요청 수를 제한하는 정책. GitHub REST 도 미인증 60/시, 인증 5000/시 등 한도가 있음
실패 편지함	Dead Letter Exchange (DLX)	Consumer 가 처리에 실패하거나 TTL 이 지난 메시지를 다른 Exchange 로 보내 격리·재처리에 쓰는 구성. `x-dead-letter-exchange` 인자로 설정
재시도 한도	Prefetch / 수동 Ack	Consumer 가 한 번에 받을 수 있는 메시지 수(`prefetch`) 와, 처리 성공 시에만 Ack 를 보내는 설정. 처리 실패 시 메시지를 Requeue 할지 DLX 로 보낼지 결정에 영향
같은 상자 두 번 와도 결과 같게	멱등성 (Idempotency)	같은 메시지가 중복으로 도착해도 최종 상태가 달라지지 않는 설계 원칙. 내부적으로 처리한 키를 기록하거나, 결과가 같은 연산 (UPSERT) 으로 구현

재시작해도 큐가 살아있음	durable Queue	브로커 재시작 후에도 Queue 메타가 유지되는 설정. 메시지까지 디스크에 보관되려면 `persistent delivery` 로 발행해야 함
Polling 주기 자동화	`@Scheduled(fixedRateString = ...)`	Spring Scheduling. 고정 간격으로 메서드를 반복 실행. `fixedRateString` 은 문자열 기반으로 프로퍼티 주입이 가능

이것만은 기억하자

- **생산자와 소비자 사이를 끊어 놓는 것이 핵심이다.** 두 서비스가 REST 로 직접 묶이면 한쪽의 장애·지연이 다른 쪽에 그대로 전파됩니다. 가운데에 메시지 브로커가 끼면 송신자는 벨트에 올려 두고, 수신자는 자기 페이스로 집어 갑니다. 장애 전파·이벤트 손실·속도 결합 세 문제가 같이 풀립니다
- **Exchange · Queue · Routing Key 세 이름은 반드시 양쪽이 같은 값을 바라본다.** Direct Exchange 는 Routing Key 가 한 글자만 달라도 메시지가 도착하지 않습니다. 실습 처음에 세 이름을 `application.properties` 에 박아 두고, Producer · Consumer 가 같은 값을 참조하게 합니다
- `convertAndSend` **한 줄과** `@RabbitListener` **한 줄이 양 끝이다.** Producer 는 `rabbitTemplate.convertAndSend(exchange, routingKey, payload)` 한 줄로 AMQP 프로토콜을 신경 쓰지 않고 객체를 JSON 으로 보내고, Consumer 는 `@RabbitListener(queues = "...")` 한 줄로 Queue 를 구독합니다. 중간의 직렬화는 `Jackson2JsonMessageConverter` 가 맡아서, 양쪽 DTO 의 필드 이름만 같으면 통과합니다
- **폴링은** `sha` **비교 +** `lastSha` **기억이 필수다.** sha 는 Git 에서 가장 확실한 변경 식별자이고, `lastSha` 를 기억하지 않으면 같은 변경을 반복 발행합니다. 운영에서는 `lastSha` 를 Redis 같은 외부 저장소에 보관해 재시작·다중 인스턴스에서도 상태가 유지되게 합니다
- **Polling 과 Webhook 은 앞단만 바뀌고 뒷단은 재사용된다.** Polling 은 우리가 나가서 확인, Webhook 은 GitHub 가 먼저 알려옴. 앞단의 진입 방식이 달라도 Exchange · Queue · Consumer 는 그대로입니다. 가운데를 단단히 만들어 두면 외부 연결 방식이 바뀌어도 내부는 흔들리지 않습니다
- **여섯 기둥으로 서비스가 닫혔다.** Docker 로 환경을 맞추고(1장), OAuth · Redis 로 인증과 세션을 얹고(2·3장), S3 Presigned URL 로 파일을 떼어 내고(4·5장), SSE · WebSocket · HLS 로 실시간·스트리밍을 붙이고(6·7장), Elasticsearch 로 검색을 빠르게 하고(8장), RabbitMQ 로 이벤트를 비동기로 분리(9장). 처음엔 이름만 들어 보던 여섯 기둥이 이제 자기 서비스 안에서 각자의 이유로 박혀 있는 모습입니다